

```

| c1 |
+----+
| 2 |
| 3 |
| 4 |
+----+

mysql> INSERT INTO t1 VALUES(0);

mysql> SELECT c1 FROM t1;
+----+
| c1 |
+----+
| 2 |
| 3 |
| 4 |
| 5 |
+----+

```

InnoDB AUTO_INCREMENT Counter Initialization

This section describes how [InnoDB](#) initializes [AUTO_INCREMENT](#) counters.

If you specify an [AUTO_INCREMENT](#) column for an [InnoDB](#) table, the in-memory table object contains a special counter called the auto-increment counter that is used when assigning new values for the column.

In MySQL 5.7 and earlier, the auto-increment counter is stored in main memory, not on disk. To initialize an auto-increment counter after a server restart, [InnoDB](#) would execute the equivalent of the following statement on the first insert into a table containing an [AUTO_INCREMENT](#) column.

```
SELECT MAX(ai_col) FROM table_name FOR UPDATE;
```

In MySQL 8.0, this behavior is changed. The current maximum auto-increment counter value is written to the redo log each time it changes and saved to the data dictionary on each checkpoint. These changes make the current maximum auto-increment counter value persistent across server restarts.

On a server restart following a normal shutdown, [InnoDB](#) initializes the in-memory auto-increment counter using the current maximum auto-increment value stored in the data dictionary.

On a server restart during crash recovery, [InnoDB](#) initializes the in-memory auto-increment counter using the current maximum auto-increment value stored in the data dictionary and scans the redo log for auto-increment counter values written since the last checkpoint. If a redo-logged value is greater than the in-memory counter value, the redo-logged value is applied. However, in the case of an unexpected server exit, reuse of a previously allocated auto-increment value cannot be guaranteed. Each time the current maximum auto-increment value is changed due to an [INSERT](#) or [UPDATE](#) operation, the new value is written to the redo log, but if the unexpected exit occurs before the redo log is flushed to disk, the previously allocated value could be reused when the auto-increment counter is initialized after the server is restarted.

The only circumstance in which [InnoDB](#) uses the equivalent of a [SELECT MAX\(ai_col\) FROM *table_name* FOR UPDATE](#) statement to initialize an auto-increment counter is when [importing a table](#) without a `.cfg` metadata file. Otherwise, the current maximum auto-increment counter value is read from the `.cfg` metadata file if present. Aside from counter value initialization, the equivalent of a [SELECT MAX\(ai_col\) FROM *table_name*](#) statement is used to determine the current maximum auto-increment counter value of the table when attempting to set the counter value to one that is smaller than or equal to the persisted counter value using an [ALTER TABLE ... AUTO_INCREMENT = *N* FOR UPDATE](#) statement. For example, you might try to set the counter value to a lesser value after deleting some records. In this case, the table must be searched to ensure that the new counter value is not less than or equal to the actual current maximum counter value.